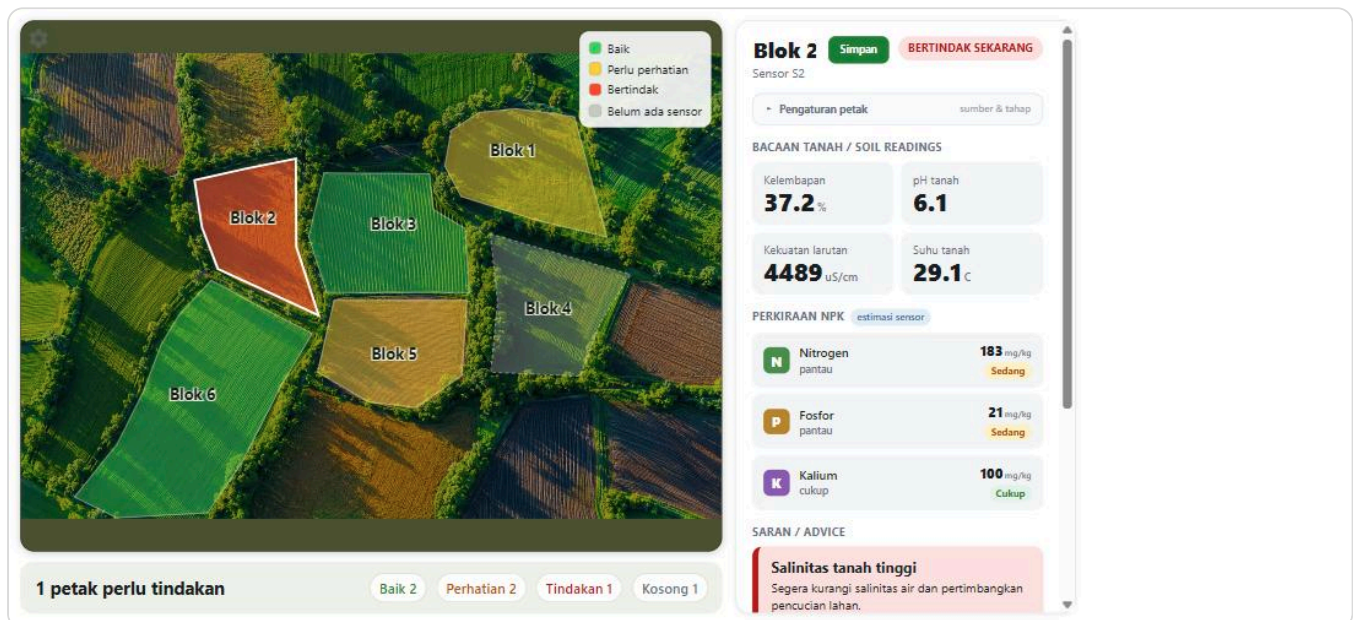


RURAL AI - NEWCASTLE X UNIVERSITAS HASANUDDIN

SawahPintar

Deployment Manual - building the kit and running a workshop



The farmer display: a map of the field, one plot per sensor.

For Newcastle and Hasanuddin faculty building kits and running soil-sensor teaching workshops for rice farmers in South Sulawesi.

This document is generated from docs/deployment-manual.md.

For Newcastle and Hasanuddin faculty building kits and running workshops. This manual assumes you are comfortable with a Windows laptop and a command line, but not that you are a Python developer.

1. What the kit is

SawahPintar is a soil sensor teaching tool for rice farming workshops. One laptop, one soil probe and one RS485-to-USB dongle make up one kit. A faculty facilitator drives the laptop; farmers get a hands-on moment pushing the probe into the soil and watching the screen respond.

The application runs entirely offline. It needs no installation, no administrator rights and no internet connection once built. Everything the laptop needs, including the Python runtime itself, travels inside the program folder.

Each kit is a complete, independent set. If a workshop runs four focus groups at once, you need four laptops, four probes and four dongles, each running its own copy of the folder. Sets never talk to each other and never need to.

What is in a built kit:

- A laptop running Windows, with Microsoft Edge already installed (it ships with Windows, so this is normally already true).
- One SN-3002-TR five-pin soil multi-parameter probe.
- One RS485-to-USB dongle with its driver installed on that laptop.
- The `SawahPintar` program folder, copied from a USB stick or a network share, containing the bundled Python runtime and the application.

Replication is copying the folder. There is no installer to run and no licence key to enter.

2. Building a kit from scratch

This is the section that must be followed exactly. One missing step produces a kit that looks fine and then fails the moment a dependency is imported. Build on a machine with internet access; the workshop laptop itself never needs one.

2.1 What you need before you start

- A Windows machine with internet access and PowerShell.
- This source repository, checked out or copied to that machine.
- Roughly 15 minutes and about 100 MB of free disk space for the build.

2.2 Run the build script

From the repository root, in PowerShell:

```
scripts\build_portable.ps1
```

This does everything: downloads the Python runtime, patches it, installs pip, installs the six dependencies, and copies the application and its data into a folder named `SawahPintar` next to the script. You can point it at a different output folder with `-OutputDir`, and pin a different Python patch version with `-PythonVersion`, though there is no reason to change either for a normal build.

Do not run this script on the workshop laptop. It needs internet access that the workshop laptop will not have, and its job is to produce the folder you then carry to that laptop offline.

2.3 What the script does, and why each step is proven, not guessed

Every fact in this section was verified by actually building and testing the runtime, not assumed from documentation. Treat all of it as load bearing.

Step 1: download the embeddable Python runtime. The script fetches

`https://www.python.org/ftp/python/3.12.8/python-3.12.8-embed-amd64.zip`, which is 11,094,114 bytes.

This is the official "embeddable" build: extract the zip and the interpreter runs immediately, with no installer and no administrator rights. This is what makes a no-install kit possible at all.

Step 2: fix the trap. This is the single most important line in this manual. The embeddable zip ships a file called `python312._pth` with the line `#import site` commented out. If that line stays commented out, the interpreter cannot see anything installed into site-packages, which means every one of the six dependencies below will fail to import even though pip installed them without complaint. The symptom is confusing: pip reports success, the packages sit on disk, and the application still crashes on its first import. The fix is to uncomment that line so it reads `import site`. The build script does this automatically with a text replace, but if you are ever troubleshooting a kit that behaves this way, this file is the first thing to check. One commented-out line is the difference between a working kit and a dead one.

Step 3: bootstrap pip. The embeddable distribution does not include pip. The script downloads `get-pip.py` from the official bootstrap URL and runs it against the newly patched interpreter, which installs pip into the runtime.

Step 4: install the seven dependencies. All seven are pure Python wheels, meaning no compiler and no build tools are needed on the workshop laptop, and all seven have been verified importable on 3.12.8:

Package	Purpose
duckdb	the readings database, a single file with no server to run
pyserial	RS485 serial communication with the probe
fastapi	the local web application that serves the interface
uvicorn	the server that runs the FastAPI application
websockets	the live feed from the server to the display, see below
pyyaml	reads the rule, content and calibration files
pytz	see below, do not skip this one either

Two of these are required but easy to miss, for the same reason. Neither pytz nor websockets is imported by the application's own code, so a casual reading of the source would not reveal that they are needed, and neither is pulled in automatically by the package that actually needs it. On a development machine both are usually already present because some other software dragged them in, so the gap is invisible there. Every workshop laptop starts clean, so it hits the gap every time. Both were found the same way: by running against the actual embeddable runtime, not the development machine. If anyone ever proposes trimming either one because "nothing imports it", point them at this paragraph first.

- **pytz.** duckdb needs it internally to read TIMESTAMPTZ columns back out of the database, but does not declare it. Without it, 11 tests fail with `Required module 'pytz' failed to import`, and the application cannot read its own readings back.
- **websockets.** uvicorn needs it to upgrade the live feed connection, but ships no WebSocket implementation of its own. Without it, the `/ws` connection returns 404, no live readings ever reach the display, and the screen sits behind a permanent "reconnecting" banner showing only its opening snapshot. The tell in the server window is the line "No supported WebSocket library detected". The automated tests do not catch this on their own, because the test client has its own WebSocket support that the real server does not use, which is exactly why the build must be tested by starting the real server, not only by running the unit tests.

Step 5: copy the application. The script copies the `app/` and `data/` folders (excluding the working database file, so a fresh one gets created on first run), `config.json`, and `Start Workshop.bat` into the output folder. The build script itself is never copied in; it has no place on the workshop laptop.

2.4 Size the USB stick correctly

The assembled runtime, with all six dependencies installed, comes to about 92 MB on disk. An earlier estimate of 45 MB was wrong; use 92 MB as your planning figure, and round up generously if you are copying several kits from one stick or burning kits onto smaller media.

2.5 Certification rule: never certify from a developer machine

Before a kit goes to a workshop, always build it from scratch on a clean process and run the full automated test suite against that exact runtime, not against whatever Python is already installed on your laptop. This is not a formality. It is exactly how the pytz gap above was caught: the suite passed happily on a development machine that already had pytz present, and only failed when run against the real embeddable runtime that a workshop laptop would actually have. A kit that has never been tested against its own runtime is unverified, however confident it looks.

2.6 Copy to the workshop laptop

Copy the whole `SawahPintar` folder to a USB stick, then to the workshop laptop, by any normal means: USB stick, network share, or external drive. Nothing in the folder needs to run again once copied; it is ready to start.

3. Hardware setup

3.1 Wiring the probe

The SN-3002-TR is a five-pin sensor speaking Modbus RTU over RS485. Four wires carry power and data:

Wire colour	Connects to
Brown	Power positive, 4.5 to 30 V DC
Black	Ground
Yellow	RS485-A
Blue	RS485-B

Getting A and B swapped is the most common wiring mistake. The probe will not respond, or will respond with garbled data, if A and B are reversed. If a freshly wired probe gives no response, check this before anything else.

Communication settings, fixed in the probe and in the sensor profile, are 4800 baud, 8 data bits, no parity, 1 stop bit, at Modbus slave address 0x01. You do not need to set these anywhere; they are already correct in `data/profiles/sn3002.json`.

3.2 Connecting the RS485-to-USB dongle

Plug the dongle into the laptop before starting the application. Windows should recognise it and assign it a COM port automatically if the dongle driver (commonly CH340 or FTDI, depending on the dongle brand) is already installed. If Windows does not recognise the dongle at all, install the matching driver first; this is the one external dependency the kit cannot bundle, because it depends on the specific USB chip inside your dongle.

3.3 Finding the right COM port

Open Windows Device Manager and look under "Ports (COM & LPT)" while the dongle is plugged in. The dongle will appear as something like "USB-SERIAL CH340 (COM9)". The number after COM is the port you need.

The COM port is set inside the application itself, not in a separate console. When a plot is set to Langsung (live), a Port box appears for it, either in the plot detail's "Pengaturan petak" panel (section 4.2) or in the zone editor (section 4.2a), where you enter the port number Device Manager reported above.

3.4 What to do when the port is wrong

If the port shown in `config.json` does not match the dongle's actual port, or if you move the dongle to a different USB socket and Windows assigns it a new number, the sensor will not respond. Two ways to fix this:

- Set the plot to Langsung (live) and enter the correct port in its Port box, either in the plot detail's "Pengaturan petak" panel (section 4.2) or in the zone editor (section 4.2a). The change saves automatically to `config.json`, so it holds for next time.

- Or edit `config.json` directly: change the `port` value under `sensors.probe-a` to the correct COM port, then restart the application.

Each dongle keeps the same COM port on the same laptop and the same USB socket, so once a kit is set up correctly it should not drift between sessions unless the dongle moves to a different port.

4. Running a workshop

4.1 Starting the app

Double-click `Start Workshop.bat` inside the kit folder. This script:

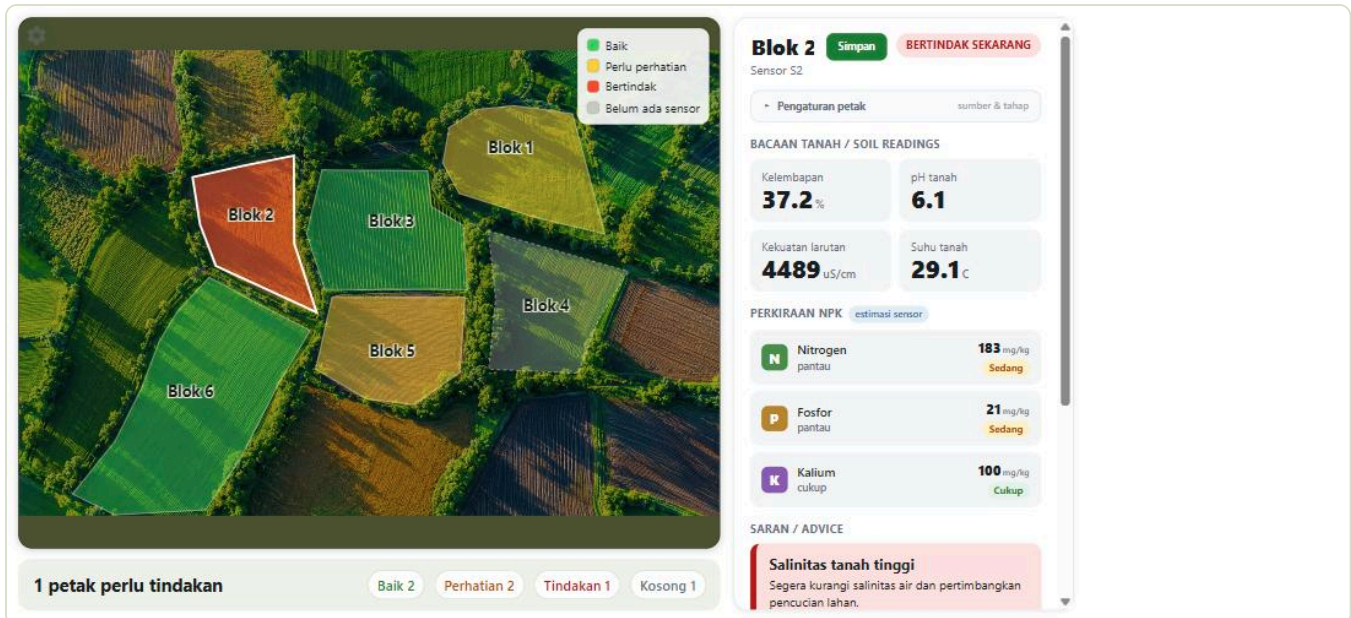
1. Confirms the bundled Python runtime is present.
2. Locates Microsoft Edge the way Windows itself does, by reading the registry entry Edge writes when it installs, which records its real location whatever the drive or folder. If that is missing it falls back to the standard folders, built from the Windows environment variables so a laptop whose Windows is not on the C: drive still resolves. If no Edge is found at all, it opens the app in the default browser instead of failing, so the kit still works, just without the clean full-screen window.
3. Starts the application server in the background, listening on `http://127.0.0.1:8420`, in simulation mode.
4. Waits for the server to actually be ready, then opens Edge in application mode (`--app=http://127.0.0.1:PORT`), which gives a clean window with no address bar, tabs or browser furniture. This is what farmers see: it reads as a purpose-built device, not a web page. The first run takes longer, around half a minute, because it prepares the demonstration history; the launcher waits for that rather than opening the window onto a half-ready screen.

The launcher always starts in simulation mode. That is the safe default for the demonstration: the seeded history is exactly what the walkthrough uses, and the facilitator switches a plot to live during the session when a real probe is connected, as described in section 4.2. It also means a live binding left in the configuration from a previous session can never leave a kit stuck on a dead feed at startup.

If the window opens to a blank or error page on a slow laptop, wait a few seconds and reload; the server may still have been finishing its first-run setup.

4.2 The farmer display

This is the window that opens automatically and is the only thing farmers should see. It is designed to be read by a standing group in daylight: large type, high contrast, a light palette. When the field has been laid out as plots (section 4.2a), it takes the form of a field map, sized to fit one screen with no page scrolling. There is no "SIMULASI" watermark any more; whether a plot is simulated or live is shown inside that plot's own detail, not stamped across the whole screen.



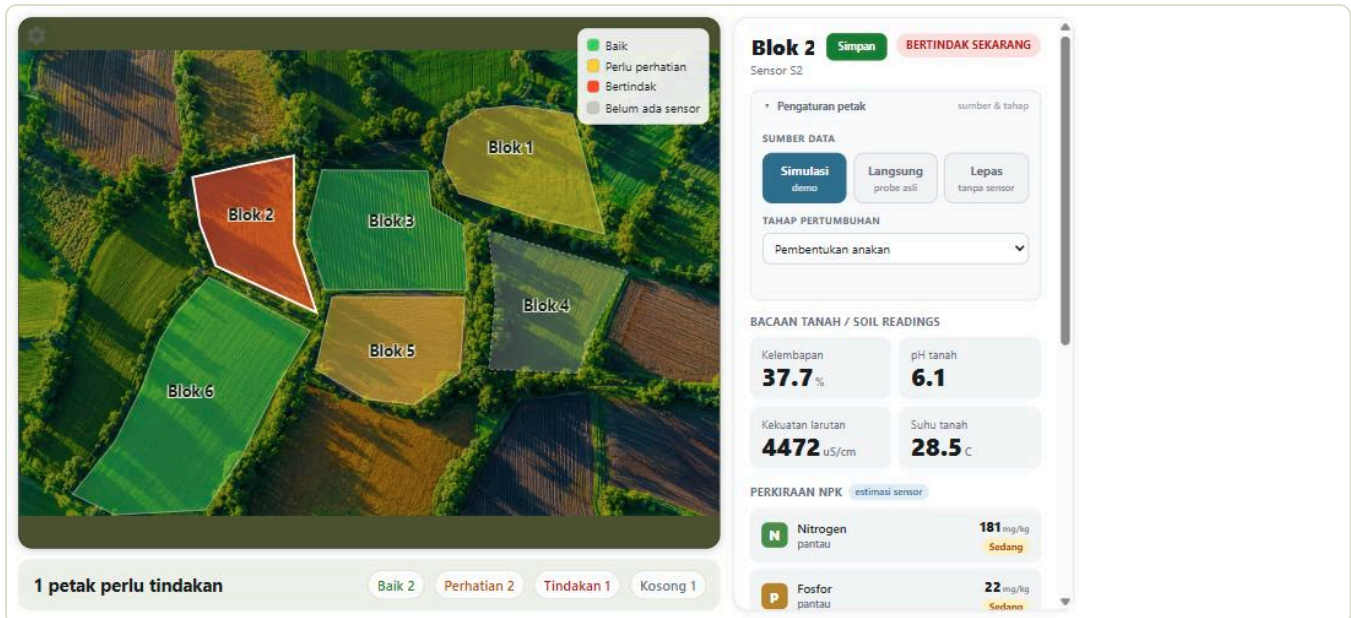
The farmer display: the field map on the left, the selected plot's detail on the right.

The farmer display. The aerial field photo and its plots sit on the left; the selected plot's detail is on the right.

The map. The field photo carries one outlined plot per sensor, each tinted by its alert colour: green for Baik (satisfactory), amber for Perlu perhatian (needs attention), red for Bertindak (act now), and a dashed grey outline for "Belum ada sensor", a plot that has been drawn but has no sensor attached. A colour legend sits at the top-right of the map. Below the map is a summary bar: a headline such as "1 petak perlu tindakan" (one plot needs action) and counts across the four states, Baik, Perhatian, Tindakan and Kosong (empty). The plots recolour live as the readings change.

A plot's detail. Tapping a plot opens its detail on the right. From top to bottom it holds:

- The plot name in an editable text box, a green "Simpan" (save) button beside it, and a status pill.
- A collapsible "Pengaturan petak" (plot setup) panel, closed by default so it stays out of the farmers' way. Opening it reveals two controls: "Sumber data" (data source), three buttons Simulasi / Langsung / Lepas (simulation, live probe, detached) with a Port box shown when the plot is live; and "Tahap pertumbuhan" (growth stage), a per-plot dropdown.
- "Bacaan tanah" (soil readings): the four probe readings, Kelembapan (moisture, per cent), pH tanah (soil pH), Kekuatan larutan (solution strength, conductivity in uS/cm) and Suhu tanah (soil temperature, C).
- "Perkiraan NPK" (NPK estimate), carrying an "estimasi sensor" tag: three rows, N, P and K, each with a value in mg/kg, a band tag (Rendah, Sedang or Cukup) and a light suggested action such as pantau, cukup or pertimbangkan urea. This is a sensor estimate, never a dose; section 7.2 explains the honesty position behind it.
- "Saran" (advice): up to four advice cards, ranked by severity then topic.



A plot's detail with the "Pengaturan petak" panel expanded.

A plot's detail with "Pengaturan petak" opened, showing the data-source buttons and the growth-stage dropdown.

Editing is immediate, and it sticks. Renaming a plot, type into the box then click Simpan or press Enter, updates the plot's label on the map at once and saves it to `config.json`. Changing the data source or the growth stage saves on its own, with a small "Tersimpan" (saved) cue. So the operator's setup survives the next launch rather than having to be redone.

When a probe goes quiet. If a live plot's probe stops answering, a "Sensor tidak merespons" (sensor not responding) banner appears, but only after about 45 seconds of continuous silence. It no longer flashes on a single slow read, so an occasional slow poll does not alarm the room.

Fallback: a single bare probe. If no plots have been drawn, one probe and no zones, the farmer display shows the original panel view instead: an overall status indicator, up to four advice cards, dials for moisture, pH, conductivity and temperature, and a trends panel collapsed by default. Draw even one plot (section 4.2a) and the display switches to the field map.

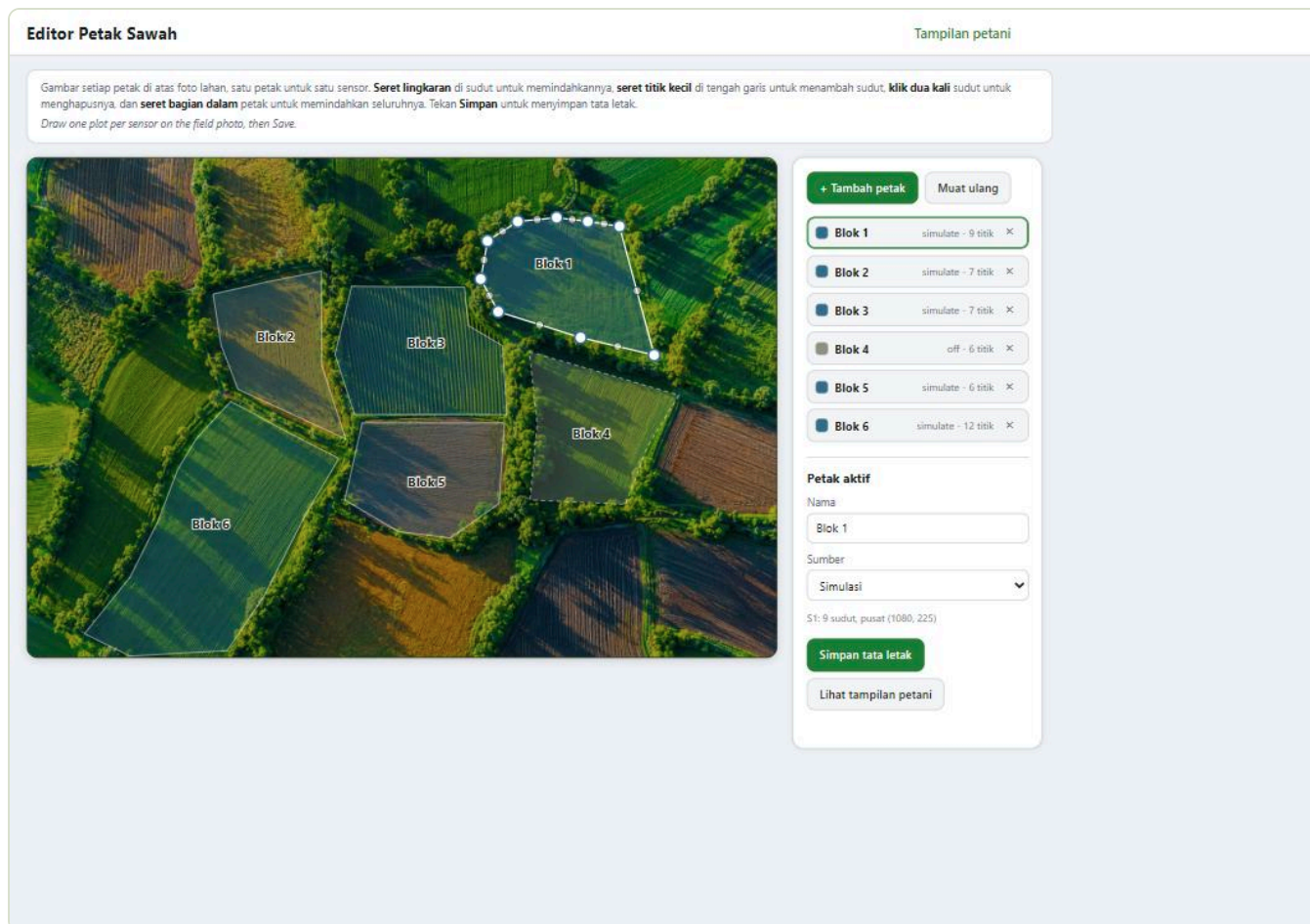
4.2a Laying out the field (the zone editor)

The field map is built once per site in the zone editor. Open it from the gear icon in the top-left corner of the farmer display, or go straight to `/zone-editor.html`. You draw one plot per sensor over the field photo.

Shaping a plot uses direct handles on the photo:

- Drag a corner handle to move that corner.
- Drag a small mid-edge dot to add a new corner there.
- Double-click a corner to remove it.
- Drag a plot's interior to move the whole plot at once.

For each plot, set its Nama (name), its Sumber (Simulasi, Langsung or Lepas) and, when live, its Port. Add a plot with "+ Tambah petak" and remove one with the small x on it. When the layout looks right, press "Simpan tata letak" (save layout); the farmer map then shows it immediately. The editor also links across to the farmer view and to the operator console, shown as "Konsol operator (PUTS, ekspor)".



The zone editor: plots drawn over the field photo, each with its name, source and port.

The zone editor. Each sensor gets one plot traced over the field photo, with its name, source and port set alongside.

The bundled photo is a stand-in. A real site replaces `app/web/assets/field-default.jpg` with a drone or satellite image of the actual field and re-traces the plots against it.

4.3 The operator console

The operator console is no longer the main settings screen. The controls a facilitator reaches for most, a plot's data source, its name and its growth stage, now live on the farmer display itself: in the plot detail (section 4.2) and in the zone editor (section 4.2a). The console holds what did not move there.

Open it from the plot editor, using its "Konsol operator (PUTS, ekspor)" link, or press `Ctrl+Alt+O` from anywhere. It opens in a separate view that farmers never see; it is dense and built for the facilitator. From here you can:

- Use the register inspector to read arbitrary Modbus registers directly, useful for confirming a new sensor's layout in the field (see section 5.3).
- Set the site name, and toggle the interface language between Bahasa Indonesia and English.
- Fill in the field card: field size, variety, seedling age, water source, previous yield and available fertiliser.
- Record a PUTS result (nitrogen, phosphorus and potassium status class, plus pH). This is the only path to a numeric fertiliser dose; see section 7.

- In simulate mode, trigger the "insert probe" and "withdraw probe" scenario buttons to rehearse the insertion moment before farmers arrive.
- Reset the demo, which clears live readings and the field card so a new focus group starts clean without touching the seeded history.
- Export the session.

4.4 Growth stage

Growth stage is a per-plot dropdown, set in each plot's "Pengaturan petak" panel on the farmer display (section 4.2): land preparation, transplanting, tillering, panicle initiation, flowering, or ripening. Set it for a plot at the start of a session and change it if the group discussion moves to a different point in the season. It matters because two rule groups read it directly: salinity severity is escalated during panicle initiation and flowering, and water advice is suspended and replaced with a "keep the field flooded" message during those same two stages, regardless of what the water table reading says.

4.5 The field card

A short form in the operator console, filled in once per focus group. It captures the inputs a fertiliser recommendation service such as Rice Crop Manager needs and no probe can supply: field size, rice variety, seedling age, water source, previous yield and fertiliser locally available. This is what makes the fertiliser guidance honestly site-specific rather than implying that a probe reading alone can produce it.

4.6 PUTS entry

PUTS (Perangkat Uji Tanah Sawah) is Indonesia's national colorimetric soil test kit for paddy fields. If the group has run a PUTS test, enter the resulting nitrogen, phosphorus and potassium status classes (rendah, sedang or tinggi) and the pH reading into the operator console. This is the only path by which the application will show a numeric fertiliser dose; see section 7 for why.

4.7 The probe moment

This is the emotional centre of the workshop and is treated as a first-class feature in the interface. Before the probe goes into the soil, moisture and conductivity read at or near zero and the overall indicator typically shows red. The moment the probe is pushed into wet soil, a step-change detector on the server notices the sharp jump in moisture and conductivity, plays a short reading animation, and then reveals the fresh assessment. Farmers watch the screen react to their own hands in real time, which is the point of the whole exercise.

4.8 Export at the end

Press "Ekspor data (Parquet dan CSV)" in the operator console at the end of a session. This writes a Parquet and a CSV file of the session's readings into `data/exports/`, both built from the same query so either university can open the data in whichever tool it prefers. The file names carry a UTC timestamp and the site name, for example `readings_20260723T050000Z_Blok_A.parquet`, specifically so that a second focus group's export cannot silently overwrite the first group's; every export lands at its own path.

If a PUTS result was recorded during the session, export it every time. Unlike the probe readings, the paired PUTS-and-probe observation

- the PUTS result together with the concurrent probe reading, the site, the date and the growth stage - lives only in `data/workshop.duckdb`, which is not part of the portable build and is not copied when a laptop is refreshed from the master folder. It does **not** persist independently of this export. When any PUTS result exists, the same button also writes `puts_observations_<timestamp>_<site>.parquet` and `.csv` alongside the readings files. This paired dataset, a validated PUTS result beside the probe's own reading of the same soil, is the one thing the kit produces that cannot be reconstructed if it is lost, and it is the data both universities most want out of the workshops. A short `README.txt` is written into `data/exports/` alongside every export, carrying the nutrient caveat from section 7 on what the probe's nitrogen, phosphorus and potassium columns actually are, so the caveat travels with the data wherever it is copied.

Copy the exported files off the laptop before the next focus group runs "Atur ulang demo" (reset), since that action clears live readings.

5. What faculty can change without a developer

Three files are designed to be opened and edited directly, in a text editor, by someone who has never written Python. All three are YAML, which is plain indented text, not code.

5.1 Rule thresholds: `data/rules/default.yaml`

This file defines every condition that triggers an advice card: the salinity classes, the water re-flood trigger, the pH 5.5 acidity threshold, and the nutrient bands. Each rule has a plain structure: an id, a group, a severity, a content key, and a condition to test. To change a threshold, for example moving the acidity trigger from pH 5.5 to a different value, edit the `value` field on the relevant condition and save. No restart of the build is needed, only a restart of the application.

Each rule group in the file carries a comment citing where its threshold came from. Keep those comments intact if you edit the numbers, so the next person can tell whether a value is still the cited figure or a locally adjusted one.

5.2 Bilingual content strings: `data/content/advice.yaml`

This is the wording farmers actually read: a Bahasa Indonesia headline and body for every advice card, plus an English subtitle for academic visitors, and an icon key. Edit the `headline_id` and `body_id` fields to change what a farmer sees; edit `subtitle_en` for the English gloss.

Every entry in this file currently carries `draft: true`, and this is deliberate. Nothing in the wording has yet been reviewed by Hasanuddin faculty for accuracy, tone, or fit with local usage, and it may need Bugis or Makassarese terms in places rather than standard Bahasa Indonesia. Treat every string as a placeholder until that review happens, not as finished copy. Once a string has been reviewed and confirmed, the `draft` marker should be removed so the two states stay distinguishable.

5.3 Calibration values: `data/calibration/default.yaml`

This file maps raw sensor numbers onto the categories the rules use: bulk conductivity in microsiemens per centimetre onto a salinity class, and volumetric moisture percentage onto an estimated water-table depth. It carries a `provisional: true` marker at the top, and that marker is accurate: neither mapping has been checked against local South Sulawesi paddy soils. The breakpoints are illustrative, not measured. Section 7 explains what to do about this and why it matters more than it might first appear.

If a different vendor's sensor needs to be added, that goes in a fourth file: create a new JSON profile in `data/profiles/`, following the shape of `sn3002.json`, describing the new probe's baud rate, register map and scaling. This needs no code change either. If the new sensor's register layout is not already documented, use the register inspector in the operator console (section 4.3) to poll it directly and work out the mapping in the field.

6. Troubleshooting

Symptom	Likely cause	What to do
<code>Start Workshop.bat</code> says it cannot find Python	The <code>python</code> folder is missing from the kit folder	Re-copy the assembled <code>SawahPintar</code> folder; do not copy only <code>app</code> and <code>data</code>
The app opened in an ordinary browser with tabs, not a clean full-screen window	No Microsoft Edge was found, so the launcher fell back to the default browser	The kit still works this way; to get the clean window, install Microsoft Edge and run again
Every dependency import fails on a freshly built kit	<code>python312._pth</code> still has <code>import site</code> commented out	Open that file inside <code>python/</code> , confirm it reads <code>import site</code> with no leading <code>#</code> , rebuild if not
The suite passed on the build machine but the kit still fails imports at the workshop	The kit was certified against a developer machine's Python, not the actual embeddable runtime	Rebuild from scratch and re-run the suite against the real 3.12.8 runtime before shipping again
Sensor shows no reading, or a reconnect banner appears	Wrong COM port, or the dongle driver is not installed	Check Device Manager or the plot's Port box in "Pengaturan petak" (section 4.2); reinstall the CH340 or FTDI driver if the dongle is unrecognised
Sensor reads garbled or implausible values	RS485 A and B wires reversed	Check wiring against section 3.1; yellow is A, blue is B
Application opens but the browser window has address bars and tabs	Edge opened normally rather than in application mode, usually because the script could not find it and something else launched instead	Confirm Edge is at one of the two standard paths; close the window and re-run <code>Start Workshop.bat</code>
Farmer display never updates from zero after the probe goes in	Probe not fully seated in moist soil, or in simulate mode with no scenario triggered	Reseat the probe; if running a rehearsal, use the operator console's "Masukkan probe" scenario button
A new focus group sees the previous group's readings	Demo was not reset between groups	Press "Atur ulang demo" in the operator console before the next group starts, after exporting the previous group's data
Exported files are missing after a session	Export was not run before "Atur ulang demo" was pressed	Always export before resetting; reset clears live readings and cannot be undone
The kit is showing simulated readings, not the real probe	The kit always starts in simulation, which is the safe default	Open the plot, expand "Pengaturan petak" and set its Sumber to Langsung (live), then enter the port (section 4.2). To rehearse without a probe, leave the plot on Simulasi

Symptom	Likely cause	What to do
The display shows "Sensor tidak merespons" (sensor not responding) on a live plot	The probe cannot be read: not connected, wrong port, or the port is held by another program. The banner waits about 45 seconds of continuous silence before it appears	Check the probe is plugged in and the port is right in "Pengaturan petak" (section 4.2); or set the plot's Sumber back to Simulasi, which recovers immediately. The feed stays alive throughout, so no restart is needed
The display is stuck on "Sambungan terputus" (reconnecting) and never shows live data, even in simulation	The bundled runtime is missing its WebSocket library, so the live feed cannot connect	Rebuild the kit; the build script installs websockets. The server window will also show "No supported WebSocket library detected". See section 2.3, step 4

7. The honesty position

This section exists so that any facilitator can answer, confidently and correctly, a farmer or a reviewer who asks what this probe actually measures.

7.1 What the probe's nutrient registers really are

The SN-3002-TR reports nine values in one poll, including nitrogen, phosphorus and potassium. It is tempting to read these as three independent nutrient measurements. They are not, and this is a structural fact about the instrument, not a limitation of this application. Three independent lines of evidence agree:

1. The manufacturer's own documentation describes the nitrogen, phosphorus and potassium registers, when uncalibrated, as holding f1, f2 and f3: three functions of one underlying conductivity measurement.
2. The writable calibration registers for each nutrient apply only a scale and an offset, an IEEE754 coefficient pair and an integer deviation. That is three straight lines plotted against a single variable. No combination of those coefficients can separate the three outputs from each other; a soil high in nitrogen and low in potassium is indistinguishable, to this instrument, from the reverse.
3. Direct measurement confirms it. The development probe, read in air, gave conductivity 0 with nitrogen, phosphorus and potassium all reading 0 at the same instant. They move together because they are one measurement wearing three labels.

7.2 What the application therefore does

Because of the above, the application treats the probe's nitrogen, phosphorus and potassium as an **estimate, never a measurement**, and never calculates a fertiliser dose from them. On the field map, each plot's detail shows a "Perkiraan NPK" (NPK estimate) block: the three values in mg/kg, a coarse band (rendah / sedang / cukup) and a light suggested action such as "pertimbangkan urea", always under the label **"estimasi sensor"** so nobody mistakes it for a laboratory figure. Alongside it the interface still shows the conductivity-derived soil solution strength indicator and its advice card.

Showing the three estimates at all was a deliberate choice for the workshop demo (made 2026-07-29): it turns the probe's nutrient registers into a talking point about what the instrument can and cannot do, rather than hiding them. Earlier the app showed only the single soil-solution-strength indicator. Either way, the estimate is never a dose, and the caveat above is the honest answer to give when a farmer asks how accurate it is.

The only path to a numeric fertiliser dose is through an operator-entered PUTS result. PUTS classifies nitrogen, phosphorus and potassium into rendah, sedang or tinggi by a chemical extraction method, which measures plant-available nutrients rather than bulk conductivity, and the dose that follows comes from the official Permentan 13 of 2022 status-class table. Where no PUTS result has been entered, a nutrient card falls back to a comparative statement only, for example that one spot carries a stronger soil solution than another, with no dose attached.

If a farmer or reviewer asks "why can't the probe just tell me how much fertiliser to use", the honest answer is that the probe measures one physical quantity, bulk electrical conductivity, and no arrangement of that single number can be split back out into three separate nutrients. Getting an actual dose needs either a PUTS kit in the field or a laboratory soil test; the probe's role is to show soil condition in real time, not to replace either of those.

7.3 The AWD water-table figure is not a moisture reading

The IRRI Safe Alternate Wetting and Drying method re-floods a paddy when the water table, measured through a perforated observation pipe standing in the field, drops to about 15 cm below the surface. That is a distinct physical measurement from the probe's volumetric moisture percentage, and the mapping between the two in `data/calibration/default.yaml` is a provisional placeholder, marked as such in the file, not a validated conversion.

The recommended fix is practical rather than purely theoretical: stand a length of perforated PVC pipe beside the probe during workshops. This does two things at once. It demonstrates the genuine, low-cost IRRI method to farmers exactly as it is meant to be used, and it gives you a real, paired water-table reading against the probe's moisture reading at the same moment, which is the calibration data this mapping currently lacks. Section 5.3 shows where to update the mapping once enough paired readings exist.

7.4 In one sentence

This kit can show a farmer, in real time, how their soil's moisture, temperature, pH and conductivity are behaving, and can connect a conductivity-derived indicator or an entered PUTS result to established agronomic guidance; it cannot, and does not claim to, measure nitrogen, phosphorus or potassium directly, and its water-table advice is provisional until locally calibrated.